



GLOSARIO TÉCNICO

Aplicaciones e Industria de IoT

Prof. Ing. Andrés Vera Centurión — CIT FP-UNA — 2026

1. PROTOCOLOS DE COMUNICACIÓN IoT

Reglas, formatos y mecanismos que definen cómo los dispositivos IoT intercambian datos con brokers, plataformas y aplicaciones.

Término	Definición	Ejemplo Práctico en IoT
Protocolo	Conjunto de reglas y convenciones que definen cómo se transmiten, formatean e interpretan datos entre dispositivos o sistemas en una red.	<i>MQTT define que un mensaje de telemetría del ESP32 debe tener una cabecera de 2 bytes, viajar por TCP al puerto 1883, y ser enrutado por el broker según el topic.</i>
MQTT <i>Message Queuing Telemetry Transport</i>	Protocolo de mensajería ligero basado en el modelo Publicar/Suscribir, diseñado para redes inestables y dispositivos con recursos limitados. Estándar OASIS desde 2014. Versión actual: 5.0.	<i>Un sensor DHT22 en un ESP32 publica la temperatura cada 3 segundos en el topic planta/sala-srv/rack-A/temp. Un dashboard ThingsBoard se suscribe al mismo topic y muestra el valor en tiempo real.</i>
Broker MQTT	Servidor intermediario que recibe mensajes de publicadores, los filtra por topic y los reenvía a todos los clientes suscritos. Es el componente central de toda arquitectura MQTT.	<i>Mosquitto corriendo en una Raspberry Pi recibe datos de 50 sensores de temperatura y los redistribuye a 3 dashboards y un sistema de alertas por Telegram, sin que los sensores conozcan a los suscriptores.</i>
Pub/Sub <i>Publish/Subscribe</i>	Patrón de mensajería donde los emisores (publishers) envían mensajes a un intermediario (broker) sin conocer a los receptores (subscribers). El broker distribuye los mensajes según el topic. Desacopla emisor de receptor.	<i>Modelo de WhatsApp: alguien publica un mensaje en un grupo (topic) y todos los miembros suscritos lo reciben. El servidor de WhatsApp actúa como broker.</i>
Topic MQTT	Dirección jerárquica separada por barras (/) que organiza los mensajes MQTT. No se crean previamente; existen al publicar el primer mensaje. Sensible a mayúsculas.	<i>empresa/asuncion/datacenter/rack-A3/temperatura. El topic permite filtrar datos con wildcards: empresa/asuncion/+/rack-A3/# recibe datos de cualquier área, solo del rack A3.</i>
Wildcard <i>Comodín</i>	Carácter especial en topics MQTT para	<i>Suscribirse a fabrica/villeta/+/motor-01/temp para monitorear el motor-01 en todas las líneas de producción. O fabrica/villeta/# para recibir absolutamente todos los datos de la planta Villeta.</i>

	suscripciones masivas. + sustituye exactamente un nivel de la jerarquía. # sustituye todos los niveles restantes (solo al final del topic).	
QoS <i>Quality of Service</i>	Nivel de garantía de entrega de un mensaje MQTT. Tres niveles: QoS 0 (a lo sumo una vez, sin confirmación), QoS 1 (al menos una vez, con PUBACK), QoS 2 (exactamente una vez, protocolo de 4 pasos).	QoS 0 para telemetría frecuente (temperatura cada 2 seg). QoS 1 para alertas de sobrecalentamiento que no pueden perderse. QoS 2 para comandos de parada de emergencia de un motor industrial.
PUBACK	Paquete de confirmación que el broker MQTT envía al publicador en QoS 1 para confirmar que recibió el mensaje correctamente.	El ESP32 publica una alerta ALARM con QoS 1. El broker responde con PUBACK. Si el ESP32 no recibe PUBACK en el timeout, reenvía el mensaje automáticamente.
Retained Message <i>Mensaje Retenido</i>	Mensaje MQTT que el broker almacena como último valor de un topic. Se entrega inmediatamente a cualquier nuevo suscriptor que se conecte, sin esperar al próximo mensaje.	Un sensor publica temperatura cada 5 minutos con retained=true. Un técnico abre el dashboard por primera vez y ve el último valor al instante, sin esperar 5 minutos.
LWT <i>Last Will and Testament</i>	Mensaje predefinido que el dispositivo registra al conectarse al broker. Si se desconecta inesperadamente (sin enviar DISCONNECT), el broker publica este mensaje automáticamente.	ESP32 registra LWT: topic=planta/sensor-01/status, payload={"status":"OFFLINE"}. Si se corta la energía, el broker publica el mensaje y el dashboard muestra el sensor como caído.
Keep Alive	Intervalo en segundos en que el cliente MQTT debe enviar un paquete PINGREQ al broker para demostrar que sigue activo. Si no lo hace, el broker asume desconexión y publica el LWT.	Keep Alive de 60 segundos: el ESP32 envía un ping cada minuto. Si el broker no recibe ping en 90 seg (1.5x keep alive), desconecta al cliente y publica el LWT.
HTTP/REST <i>Hypertext Transfer Protocol /</i>	Protocolo de solicitud/respuesta de la web. Verbos: GET	Una API REST en ThingsBoard permite consultar el historial de temperatura: GET

<i>Representational State Transfer</i>	(leer), POST (crear), PUT (actualizar), DELETE (borrar). Cabecera de ~700 bytes. Universal pero pesado para IoT.	<i>/api/plugins/telemetry/DEVICE_ID/values/timeseries?keys=temperature devuelve un JSON con todos los datos.</i>
CoAP <i>Constrained Application Protocol</i>	Protocolo ligero tipo HTTP diseñado para dispositivos IoT muy restringidos (RFC 7252). Usa UDP, cabecera de 4 bytes. Soporta Observe para actualizaciones tipo suscripción.	<i>Un sensor de humedad del suelo con solo 8KB de RAM y batería de botón usa CoAP sobre 6LoWPAN para enviar una lectura cada hora a un gateway LoRaWAN.</i>
WebSocket	Extensión de HTTP que permite comunicación bidireccional persistente entre cliente y servidor. Mantiene la conexión abierta después del handshake HTTP inicial.	<i>Un dashboard web muestra datos de sensores en tiempo real usando WebSocket: el servidor envía actualizaciones al navegador sin que este tenga que refrescar la página.</i>
Client ID	Identificador único que cada dispositivo MQTT usa al conectarse al broker. Si dos dispositivos usan el mismo Client ID, el broker desconecta al que se conectó primero.	<i>Usar esp32-sala-servidores-rack-A3 como Client ID, no esp32. Si 10 estudiantes usan esp32 en el mismo broker público, solo uno permanecerá conectado.</i>

2. TECNOLOGÍAS DE RED

Infraestructura de comunicación que transporta los datos desde el sensor hasta la plataforma: desde redes corporales hasta redes de largo alcance.

Término	Definición	Ejemplo Práctico en IoT
TCP <i>Transmission Control Protocol</i>	Protocolo de transporte orientado a conexión que garantiza entrega ordenada y sin errores mediante confirmaciones y retransmisiones. MQTT y HTTP funcionan sobre TCP.	Cuando el ESP32 publica un mensaje MQTT, TCP garantiza que cada byte llega al broker en orden. Si un paquete se pierde, TCP lo retransmite automáticamente.
UDP <i>User Datagram Protocol</i>	Protocolo de transporte sin conexión, más rápido y liviano que TCP pero sin garantía de entrega ni orden. CoAP funciona sobre UDP.	Un sensor CoAP envía una lectura por UDP: si el paquete se pierde, no hay reintento automático. Aceptable si el sensor envía una lectura cada 10 segundos y perder una no es crítico.
TLS <i>Transport Layer Security</i>	Protocolo criptográfico que cifra la comunicación entre dos sistemas, garantizando confidencialidad, integridad y autenticación. En MQTT se usa en el puerto 8883.	Un ESP32 en producción se conecta al broker por MQTT+TLS (puerto 8883). Un atacante que intercepte el tráfico Wi-Fi solo verá datos cifrados, no las lecturas de temperatura.
DTLS <i>Datagram TLS</i>	Versión de TLS adaptada para UDP, usada por CoAP para cifrar comunicaciones en dispositivos restringidos que no pueden usar TCP.	Un sensor 6LoWPAN envía datos cifrados con DTLS al gateway. Mantiene la seguridad sin la sobrecarga de TCP.
BAN <i>Body Area Network</i>	Red de área corporal con alcance de ~2 metros. Conecta sensores y dispositivos que se llevan en el cuerpo. Estándar IEEE 802.15.6.	Un parche de monitoreo de glucosa continuo transmite lecturas por BAN al smartphone del paciente diabético cada 5 minutos.
PAN <i>Personal Area Network</i>	Red de área personal con alcance de ~10 metros. Conecta dispositivos cercanos al usuario mediante BLE, Zigbee u otras tecnologías de corto alcance.	Un smartwatch BLE envía datos de ritmo cardíaco al teléfono móvil del usuario para la app de salud.
LAN <i>Local Area Network</i>	Red de área local con alcance de ~100 metros. Conecta dispositivos dentro de un edificio o planta mediante Wi-Fi o Ethernet.	20 sensores ESP32 en una fábrica se conectan al router Wi-Fi local y envían datos MQTT al broker Mosquitto instalado en un servidor dentro de la misma planta.
WAN <i>Wide Area Network</i>	Red de área extendida con alcance de kilómetros. Incluye redes celulares (4G/5G), LoRaWAN, NB-IoT y satelital.	Sensores de humedad del suelo distribuidos en 500 hectáreas de soja en el Chaco paraguayo se conectan vía LoRaWAN a un gateway que reenvía los datos a la nube por 4G.
LPWAN <i>Low-Power Wide-Area Network</i>	Categoría de redes WAN diseñadas para IoT: largo alcance (km), bajo consumo energético (años de batería), baja tasa de datos (kbps). Incluye LoRaWAN, NB-IoT y Sigfox.	Un medidor de agua inteligente en una zona rural usa LPWAN para enviar una lectura diaria de consumo, funcionando con batería durante 8 años sin mantenimiento.
LoRa <i>Long Range</i>	Técnica de modulación de radiofrecuencia basada en espectro expandido, desarrollada por Semtech. Opera en bandas ISM sin licencia (915 MHz en América, 868 MHz en Europa).	Un módulo LoRa SX1276 transmite 50 bytes de datos de un sensor de suelo a 12 km de distancia con solo 25 mW de potencia, consumiendo menos energía que un LED.

LoRaWAN <i>Long Range Wide Area Network</i>	Protocolo de red abierto gestionado por la LoRa Alliance que define la comunicación entre dispositivos LoRa, gateways y servidores de red. Topología estrella de estrellas.	<i>Una red LoRaWAN agrícola en Paraguay: 200 sensores de campo → 3 gateways en silos elevados → servidor de red The Things Network → plataforma ThingsBoard para dashboards.</i>
NB-IoT <i>Narrowband IoT</i>	Estándar 3GPP (Release 13, 2016) para comunicación IoT de banda estrecha sobre infraestructura celular LTE existente. Alcance 1-10 km, hasta 250 Kbps, excelente penetración en interiores.	<i>Un medidor eléctrico inteligente de ANDE usa NB-IoT vía la red de Claro Paraguay para enviar consumo cada hora. No requiere infraestructura propia: usa las antenas celulares existentes.</i>
LTE-M <i>LTE Cat-M1</i>	Tecnología celular IoT del 3GPP con mayor ancho de banda que NB-IoT (hasta 1 Mbps). Soporta movilidad y handover entre celdas, ideal para tracking.	<i>Un rastreador GPS en un camión de logística usa LTE-M para enviar posición cada 30 segundos mientras se mueve entre ciudades, haciendo handover entre celdas automáticamente.</i>
BLE <i>Bluetooth Low Energy</i>	Versión de bajo consumo de Bluetooth (también llamada Bluetooth Smart). Alcance hasta 400m (BLE 5.0), tasa de 2 Mbps. Soporta mesh desde la versión 5.0.	<i>Beacons BLE en un hospital envían señales de localización. Una app en el celular del enfermero le muestra la ubicación de cada equipo médico en tiempo real (localización indoor).</i>
Zigbee	Protocolo de red mesh basado en IEEE 802.15.4 para domótica y automatización. Soporta hasta 65,000 nodos, bajo consumo, autorecuperación. Gestionado por la Connectivity Standards Alliance.	<i>500 luminarias Philips Hue en un edificio de oficinas se controlan vía Zigbee mesh. Si un nodo falla, la red se rerutea automáticamente a través de nodos vecinos.</i>
5G NR <i>5G New Radio</i>	Quinta generación de telecomunicaciones móviles (3GPP Release 15+). Tres modos: eMBB (velocidad), URLLC (latencia <1ms) y mMTC (IoT masivo, 1M dispositivos/km²).	<i>Un brazo robótico en una fábrica es controlado remotamente vía 5G URLLC con latencia de 0.5ms. El operador en Asunción controla el robot en Villeta como si estuviera al lado.</i>
ISM <i>Industrial, Scientific and Medical</i>	Bandas de radiofrecuencia de uso libre que no requieren licencia gubernamental. América: 915 MHz. Europa: 868 MHz. Global: 2.4 GHz (Wi-Fi, BLE, Zigbee).	<i>Un gateway LoRaWAN en Paraguay opera en 915 MHz ISM. No necesita licencia de CONATEL para operar, a diferencia de una estación celular que requiere espectro licenciado.</i>
3GPP <i>3rd Generation Partnership Project</i>	Organismo internacional que desarrolla los estándares de telecomunicaciones celulares: 3G (UMTS), 4G (LTE), 5G (NR), NB-IoT y LTE-M.	<i>Cuando Claro Paraguay despliega NB-IoT, implementa las especificaciones definidas por 3GPP en el Release 13, garantizando interoperabilidad con módulos de cualquier fabricante.</i>
MIMO <i>Multiple-Input Multiple-Output</i>	Tecnología que usa múltiples antenas para transmitir y recibir datos simultáneamente, mejorando velocidad y capacidad. Massive MIMO (5G) usa 64-256 antenas.	<i>Una antena 5G Massive MIMO con 128 elementos atiende simultáneamente a 200 sensores IoT en un estacionamiento inteligente, cada uno reportando ocupación en tiempo real.</i>
Network Slicing	Tecnología 5G que permite crear redes virtuales independientes sobre la misma infraestructura física, cada una con características específicas de latencia, ancho de banda y seguridad.	<i>Un operador 5G crea un slice dedicado para IoT industrial con latencia garantizada <5ms y otro slice para video de seguridad con alto ancho de banda, sobre las mismas antenas.</i>

Mesh

Red Mallada

Topología de red donde cada nodo puede retransmitir datos de otros nodos, extendiendo la cobertura y mejorando la resiliencia ante fallos de nodos individuales.

En una red Zigbee mesh de 100 sensores en un almacén, si el sensor #47 falla, los sensores vecinos (#46 y #48) retransmiten los datos de los sensores más lejanos automáticamente.

3. PLATAFORMAS IoT Y GESTIÓN DE DATOS

Software, servicios en la nube y herramientas que reciben, almacenan, procesan y visualizan los datos generados por los dispositivos IoT.

Término	Definición	Ejemplo Práctico en IoT
Plataforma IoT <i>IoT Platform / Middleware</i>	Software intermedio que conecta dispositivos físicos con aplicaciones de negocio. Gestiona conexiones, almacena datos, ejecuta reglas y expone interfaces para dashboards y APIs.	<i>ThingsBoard recibe datos MQTT de 50 sensores, los almacena en PostgreSQL, ejecuta reglas de alerta (si temp>32 enviar Telegram), y muestra todo en un dashboard web en tiempo real.</i>
ThingsBoard	Plataforma IoT de código abierto con gestión de dispositivos, dashboards visuales (drag & drop), motor de reglas y soporte MQTT/HTTP/CoAP. Edición Community gratuita.	<i>En el LAB del curso, el ESP32 publica telemetría a ThingsBoard Cloud. El estudiante crea un dashboard con gráfico de temperatura, gauge de humedad y tarjeta de estado sin escribir código.</i>
AWS IoT Core	Servicio de AWS que proporciona un broker MQTT gestionado, motor de reglas, Device Shadows y integración nativa con servicios AWS (Lambda, DynamoDB, S3). ~USD 1/millón de mensajes.	<i>Una empresa con 10,000 sensores industriales usa AWS IoT Core para ingestar datos, procesarlos con Lambda en tiempo real, y almacenarlos en S3 para análisis con Athena.</i>
Azure IoT Hub	Servicio de Microsoft Azure para gestión de dispositivos IoT. Integración nativa con Power BI, Teams, Active Directory. Incluye IoT Edge para procesamiento local y Digital Twins.	<i>Una flota de 500 vehículos usa Azure IoT Hub para telemetría GPS + OBD-II. Los datos se visualizan en Power BI y las alertas de mantenimiento llegan por Microsoft Teams.</i>
Device Shadow <i>Sombra del Dispositivo</i>	Representación JSON del estado de un dispositivo IoT almacenada en la nube. Permite consultar y modificar el estado incluso cuando el dispositivo está offline.	<i>Un sensor agrícola se desconecta por falta de señal. AWS Device Shadow guarda su último estado reported:{temp:28.5}. Cuando se reconecta, sincroniza automáticamente.</i>
Mosquitto	Broker MQTT ligero y confiable de Eclipse Foundation. Bajo consumo de recursos, ideal para servidores pequeños (incluso Raspberry Pi). El broker más usado en educación y prototipos.	<i>Un Mosquitto en una Raspberry Pi 4 de USD 35 maneja 500 sensores en un invernadero con menos de 50MB de RAM, procesando 10,000 mensajes por minuto.</i>
EMQX	Broker MQTT de alto rendimiento escrito en Erlang. Soporta millones de conexiones simultáneas, MQTT 3.1.1 y 5.0, bridge con Kafka y bases de datos.	<i>Una ciudad inteligente con 200,000 sensores de tráfico, iluminación y calidad del aire usa EMQX como broker central, manejando 5 millones de mensajes por segundo.</i>
Node-RED	Herramienta de programación visual basada en flujos (flow-based) para IoT y automatización. Conecta APIs, dispositivos y servicios mediante nodos arrastrables sin código complejo.	<i>Un flujo Node-RED recibe datos MQTT del ESP32, verifica si temp>32, y si es así envía un mensaje de alerta a un grupo de Telegram y un email al gerente de TI, todo sin escribir código.</i>
Serie Temporal <i>Time Series</i>	Secuencia de datos indexados en el tiempo. Los datos IoT son series temporales por naturaleza: cada lectura tiene un timestamp asociado.	<i>Las lecturas del DHT22 forman una serie temporal: [10:00:00, 28.1°C], [10:00:03, 28.3°C], [10:00:06, 28.2°C]... Esta serie se analiza para detectar tendencias y anomalías.</i>

InfluxDB	Base de datos de código abierto especializada en series temporales, optimizada para escritura masiva y consultas por rango temporal. Lenguaje de consulta: InfluxQL o Flux.	<i>Un sistema de monitoreo industrial almacena 1 millón de puntos por segundo en InfluxDB. La consulta <code>SELECT mean(temp) FROM sensors WHERE time > now() - 1h</code> se ejecuta en milisegundos.</i>
TimescaleDB	Extensión de PostgreSQL optimizada para series temporales. Combina la potencia de SQL estándar con optimizaciones para datos temporales. ThingsBoard la usa por defecto.	<i>ThingsBoard almacena toda la telemetría en TimescaleDB. El usuario puede consultar con SQL estándar: <code>SELECT avg(temperature) FROM ts_kv WHERE entity_id='...' AND ts > now()-interval '24h'</code>.</i>
Motor de Reglas <i>Rule Engine</i>	Componente que ejecuta lógica condicional (if-then) sobre datos entrantes en tiempo real, generando alertas, acciones o transformaciones automáticas.	<i>Regla en ThingsBoard: SI temperature > 32 Y humidity < 30 ENTONCES enviar alerta por email Y activar ventilador vía MQTT command al topic dispositivo/fan/control.</i>
Dashboard	Panel visual que muestra datos en tiempo real mediante widgets (gráficos, gauges, mapas, tablas). Permite monitorear el estado de dispositivos y KPIs sin acceder a bases de datos.	<i>Dashboard de sala de servidores en ThingsBoard: gráfico temporal de temperatura (últimas 24h), gauge de humedad (0-100%), tarjeta de estado (NORMAL/WARNING/ALARM), mapa de ubicación.</i>
Widget	Componente visual reutilizable dentro de un dashboard que muestra un tipo específico de dato: gráfico de línea, gauge circular, mapa, tabla, indicador numérico, etc.	<i>Widget Timeseries Chart de ThingsBoard: muestra la curva de temperatura en tiempo real con zoom, pan y selección de rango temporal. Se configura sin código, solo seleccionando dispositivo y variable.</i>
Datasource	Fuente de datos de un widget. En ThingsBoard es la combinación de un dispositivo y una variable (key) de telemetría.	<i>Datasource del widget gauge: dispositivo=ESP32-Sala-Servidor, key=humidity. El gauge muestra el último valor de humedad publicado por ese dispositivo específico.</i>
Access Token	Credencial de autenticación única asignada a cada dispositivo en ThingsBoard. Se usa como username en la conexión MQTT (password vacío).	<i>El ESP32 se conecta: <code>mqtt.connect('esp32-tb', 'A1B2C3D4E5F6', '')</code>. El token A1B2C3D4E5F6 identifica y autentica al dispositivo. Cada ESP32 tiene su propio token.</i>
RPC <i>Remote Procedure Call</i>	Mecanismo para ejecutar funciones en un dispositivo remoto. En ThingsBoard, permite enviar comandos al ESP32 desde el dashboard.	<i>Desde el dashboard, el gerente presiona un botón que envía RPC <code>{method:'setAlarm', params:{value:35}}</code> al ESP32. El dispositivo cambia el umbral de alarma a 35°C sin reprogramar.</i>

4. DESARROLLO Y PROGRAMACIÓN IoT

Lenguajes, librerías, herramientas y conceptos de programación utilizados para desarrollar firmware y aplicaciones en dispositivos IoT.

Término	Definición	Ejemplo Práctico en IoT
MicroPython	Implementación eficiente de Python 3 optimizada para microcontroladores. Creado por Damien George en 2013. Incluye REPL interactivo, módulos para GPIO, Wi-Fi y MQTT.	<i>Un estudiante prototipa un sensor de suelo en ESP32 con MicroPython: escribe el código en 20 líneas, lo prueba en el REPL línea por línea, y lo sube al dispositivo sin compilar.</i>
REPL <i>Read-Eval-Print Loop</i>	Terminal interactiva que ejecuta comandos de programación uno a uno, mostrando el resultado inmediatamente. En MicroPython, se accede por USB serial al microcontrolador. Permite probar código sin crear archivos ni compilar.	<i>Conectado al ESP32 por USB, en el REPL se escribe: sensor.measure() → sensor.temperature() → responde: 28.5. Se probó el sensor sin escribir un programa completo.</i>
Firmware	Software de bajo nivel grabado en la memoria no volátil (flash) del microcontrolador. En MicroPython, el firmware incluye el intérprete Python completo. En Arduino, cada sketch compilado es el firmware.	<i>Para usar MicroPython en un ESP32, primero se flashea el firmware desde micropython.org/download/esp32. Luego el ESP32 ejecuta Python directamente.</i>
PubSubClient	Librería Arduino para implementar un cliente MQTT en ESP32/ESP8266. Soporta QoS 0 y 1, LWT, conexión con usuario/contraseña. Creada por Nick O'Leary.	<i>En el código Arduino: PubSubClient mqtt(espClient); mqtt.setServer('thingsboard.cloud', 1883); mqtt.connect('esp32', token, ""); mqtt.publish(topic, json);</i>
umqtt.simple	Módulo MicroPython que implementa un cliente MQTT básico con QoS 0. Incluido en el firmware estándar de MicroPython para ESP32.	<i>from umqtt.simple import MQTTClient; client = MQTTClient('esp32', 'thingsboard.cloud', user=TOKEN); client.connect(); client.publish('v1/devices/me/telemetry', json.dumps(data))</i>
umqtt.robust	Versión mejorada de umqtt.simple que reconecta automáticamente si se	<i>Un sensor en zona rural con señal Wi-Fi intermitente usa umqtt.robust. Cuando la señal se pierde y regresa, se reconecta solo y continúa publicando sin intervención.</i>

	pierde la conexión MQTT. Ideal para dispositivos en producción con conectividad inestable.	
ADC <i>Analog-to-Digital Converter</i>	Conversor que transforma una señal analógica continua (voltaje 0-3.3V) en un valor digital discreto (0-4095 en ESP32 de 12 bits). Esencial para leer sensores analógicos.	<i>Un potenciómetro conectado al GPIO34 del ESP32 simula un sensor de corriente. El ADC lee 2048 cuando el pot está al 50%, equivalente a 1.65V.</i>
PWM <i>Pulse Width Modulation</i>	Modulación por ancho de pulso: técnica para simular salidas analógicas con pines digitales variando el porcentaje de tiempo que la señal está en HIGH (duty cycle).	<i>Controlar el brillo de un LED: PWM con duty cycle 25% = LED a 1/4 de brillo. Controlar velocidad de un ventilador: duty cycle 75% = ventilador al 75% de velocidad máxima.</i>
Deep Sleep	Modo de ultra bajo consumo del microcontrolador donde se apagan casi todos los circuitos. El ESP32 pasa de ~150mA activo a ~10µA en deep sleep (15,000x menos).	<i>Un sensor agrícola con batería solar: se despierta cada 15 minutos, lee temperatura y humedad, publica por LoRaWAN, y vuelve a deep sleep. La batería dura 2 años.</i>
GPIO <i>General Purpose Input/Output</i>	Pines de propósito general del microcontrolador que pueden configurarse como entrada (leer sensor/botón) o salida (encender LED/relé). El ESP32 tiene 34 GPIO.	<i>GPIO4 configurado como entrada para leer el DHT22. GPIO25 configurado como salida para encender el LED verde. GPIO34 configurado como entrada ADC para leer un potenciómetro.</i>
Callback	Función que se pasa como argumento a otra función para ser ejecutada cuando ocurre un evento específico. Patrón fundamental en programación de IoT.	<i>En MQTT: client.set_callback(on_message). Cuando llega un mensaje al topic suscrito, MicroPython ejecuta automáticamente la función on_message(topic, payload) con los datos recibidos.</i>
JSON <i>JavaScript Object Notation</i>	Formato de texto ligero para intercambio de datos basado en pares clave-valor. Legible por humanos y máquinas. Estándar de facto para telemetría IoT.	<i>{"device_id":"esp32-01","temperature":28.5,"humidity":62.3,"status":"NORMAL","seq":42}. En MicroPython: json.dumps(dict) para serializar, json.loads(string) para deserializar.</i>
Serverless	Modelo de ejecución cloud donde el proveedor gestiona la infraestructura	<i>AWS Lambda ejecuta una función que procesa cada mensaje MQTT: si temperature>32, envía notificación SNS. Lambda cobra solo por los milisegundos de ejecución, no por tener un servidor 24/7.</i>

Middleware	automáticamente. El usuario solo paga por el tiempo de ejecución real de su código, sin mantener servidores.	<i>ThingsBoard actúa como middleware: recibe MQTT del ESP32, almacena en PostgreSQL, ejecuta reglas, y expone APIs REST para la app móvil del gerente. Cada componente es independiente.</i>
	Software intermediario que conecta componentes de un sistema distribuido, facilitando la comunicación entre aplicaciones, bases de datos y dispositivos sin que se conozcan directamente.	
Webhook	Mecanismo de comunicación donde un servidor envía automáticamente una notificación HTTP POST a una URL configurada cuando ocurre un evento específico.	<i>ThingsBoard envía un webhook a https://api.telegram.org cuando se activa una alerta: el servidor de Telegram recibe el JSON y envía el mensaje al grupo del equipo de mantenimiento.</i>
OTA <i>Over-The-Air Update</i>	Actualización de firmware de un dispositivo IoT realizada remotamente a través de la red, sin necesidad de acceso físico al dispositivo.	<i>Se detecta un bug de seguridad en el firmware de 500 sensores distribuidos en campos agrícolas. Se envía actualización OTA vía MQTT y todos se actualizan en 30 minutos sin visitar cada uno.</i>